

# Student Course Completion Prediction

## Data Pipeline :

- Data Acquisition
- Initial Data Inspection and Cleaning
- Time-Series Analysis
- Univariate Analysis
- Bivariate Analysis
- Multivariate Analysis
- Outlier Visualization
- Outlier Analysis
- Feature Engineering for Completion Prediction
- Machine Learning
- Train and Evaluate Classification Models
- Explanation of Model Performance
- Summary

## Data Acquisition

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('D:/PYTHON 2/DATA/DECEMBER/Student Course Completion Prediction.csv')
print("Dataset loaded Successfully.")
df.head()
```

Dataset loaded Successfully.

Out[1]:

|   | Student_ID | Name            | Gender | Age | Education_Level | Employment_Status | City    |
|---|------------|-----------------|--------|-----|-----------------|-------------------|---------|
| 0 | STU100000  | Vihaan Patel    | Male   | 19  | Diploma         | Student           | Indore  |
| 1 | STU100001  | Arjun Nair      | Female | 17  | Bachelor        | Student           | Delhi   |
| 2 | STU100002  | Aditya Bhardwaj | Female | 34  | Master          | Student           | Chennai |
| 3 | STU100003  | Krishna Singh   | Female | 29  | Diploma         | Employed          | Surat   |
| 4 | STU100004  | Krishna Nair    | Female | 19  | Master          | Self-Employed     | Lucknow |

5 rows × 40 columns



# Initial Data Inspection and Cleaning

```
In [2]: print("DataFrame Info:")
        df.info()
```

DataFrame Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 100000 entries, 0 to 99999

Data columns (total 40 columns):

| #  | Column                       | Non-Null Count  | Dtype   |
|----|------------------------------|-----------------|---------|
| 0  | Student_ID                   | 100000 non-null | object  |
| 1  | Name                         | 100000 non-null | object  |
| 2  | Gender                       | 100000 non-null | object  |
| 3  | Age                          | 100000 non-null | int64   |
| 4  | Education_Level              | 100000 non-null | object  |
| 5  | Employment_Status            | 100000 non-null | object  |
| 6  | City                         | 100000 non-null | object  |
| 7  | Device_Type                  | 100000 non-null | object  |
| 8  | Internet_Connection_Quality  | 100000 non-null | object  |
| 9  | Course_ID                    | 100000 non-null | object  |
| 10 | Course_Name                  | 100000 non-null | object  |
| 11 | Category                     | 100000 non-null | object  |
| 12 | Course_Level                 | 100000 non-null | object  |
| 13 | Course_Duration_Days         | 100000 non-null | int64   |
| 14 | Instructor_Rating            | 100000 non-null | float64 |
| 15 | Login_Frequency              | 100000 non-null | int64   |
| 16 | Average_Session_Duration_Min | 100000 non-null | int64   |
| 17 | Video_Completion_Rate        | 100000 non-null | float64 |
| 18 | Discussion_Participation     | 100000 non-null | int64   |
| 19 | Time_Spent_Hours             | 100000 non-null | float64 |
| 20 | Days_Since_Last_Login        | 100000 non-null | int64   |
| 21 | Notifications_Checked        | 100000 non-null | int64   |
| 22 | Peer_Interaction_Score       | 100000 non-null | float64 |
| 23 | Assignments_Submitted        | 100000 non-null | int64   |
| 24 | Assignments_Missed           | 100000 non-null | int64   |
| 25 | Quiz_Attempts                | 100000 non-null | int64   |
| 26 | Quiz_Score_Avg               | 100000 non-null | float64 |
| 27 | Project_Grade                | 100000 non-null | float64 |
| 28 | Progress_Percentage          | 100000 non-null | float64 |
| 29 | Rewatch_Count                | 100000 non-null | int64   |
| 30 | Enrollment_Date              | 100000 non-null | object  |
| 31 | Payment_Mode                 | 100000 non-null | object  |
| 32 | Fee_Paid                     | 100000 non-null | object  |
| 33 | Discount_Used                | 100000 non-null | object  |
| 34 | Payment_Amount               | 100000 non-null | int64   |
| 35 | App_Usage_Percentage         | 100000 non-null | int64   |
| 36 | Reminder_Emails_Clicked      | 100000 non-null | int64   |
| 37 | Support_Tickets_Raised       | 100000 non-null | int64   |
| 38 | Satisfaction_Rating          | 100000 non-null | float64 |
| 39 | Completed                    | 100000 non-null | object  |

dtypes: float64(8), int64(15), object(17)

memory usage: 30.5+ MB

```
In [3]: print("Missing values per column:")
        print(df.isnull().sum())
```

```

Missing values per column:
Student_ID          0
Name                0
Gender              0
Age                 0
Education_Level     0
Employment_Status   0
City                0
Device_Type         0
Internet_Connection_Quality 0
Course_ID           0
Course_Name         0
Category            0
Course_Level        0
Course_Duration_Days 0
Instructor_Rating   0
Login_Frequency     0
Average_Session_Duration_Min 0
Video_Completion_Rate 0
Discussion_Participation 0
Time_Spent_Hours    0
Days_Since_Last_Login 0
Notifications_Checked 0
Peer_Interaction_Score 0
Assignments_Submitted 0
Assignments_Missed  0
Quiz_Attempts       0
Quiz_Score_Avg      0
Project_Grade       0
Progress_Percentage 0
Rewatch_Count       0
Enrollment_Date     0
Payment_Mode        0
Fee_Paid            0
Discount_Used       0
Payment_Amount      0
App_Usage_Percentage 0
Reminder_Emails_Clicked 0
Support_Tickets_Raised 0
Satisfaction_Rating 0
Completed           0
dtype: int64

```

## Time-Series Analysis

```
In [4]: print("Number of duplicate rows:", df.duplicated().sum())
```

```
Number of duplicate rows: 0
```

```
In [5]: df['Enrollment_Date'] = pd.to_datetime(df['Enrollment_Date'], format='%d-%m-%Y')
print("Converted 'Enrollment_Date' to datetime type.")
df.info()
```

```

Converted 'Enrollment_Date' to datetime type.
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 40 columns):
 #   Column                                          Non-Null Count  Dtype
---  -
 0   Student_ID                                    100000 non-null  object
 1   Name                                           100000 non-null  object
 2   Gender                                         100000 non-null  object
 3   Age                                             100000 non-null  int64
 4   Education_Level                               100000 non-null  object
 5   Employment_Status                             100000 non-null  object
 6   City                                           100000 non-null  object
 7   Device_Type                                   100000 non-null  object
 8   Internet_Connection_Quality                  100000 non-null  object
 9   Course_ID                                     100000 non-null  object
10   Course_Name                                   100000 non-null  object
11   Category                                       100000 non-null  object
12   Course_Level                                  100000 non-null  object
13   Course_Duration_Days                         100000 non-null  int64
14   Instructor_Rating                            100000 non-null  float64
15   Login_Frequency                             100000 non-null  int64
16   Average_Session_Duration_Min                 100000 non-null  int64
17   Video_Completion_Rate                       100000 non-null  float64
18   Discussion_Participation                     100000 non-null  int64
19   Time_Spent_Hours                            100000 non-null  float64
20   Days_Since_Last_Login                       100000 non-null  int64
21   Notifications_Checked                       100000 non-null  int64
22   Peer_Interaction_Score                      100000 non-null  float64
23   Assignments_Submitted                       100000 non-null  int64
24   Assignments_Missed                          100000 non-null  int64
25   Quiz_Attempts                               100000 non-null  int64
26   Quiz_Score_Avg                             100000 non-null  float64
27   Project_Grade                               100000 non-null  float64
28   Progress_Percentage                         100000 non-null  float64
29   Rewatch_Count                               100000 non-null  int64
30   Enrollment_Date                             100000 non-null  datetime64[ns]
31   Payment_Mode                                 100000 non-null  object
32   Fee_Paid                                     100000 non-null  object
33   Discount_Used                               100000 non-null  object
34   Payment_Amount                              100000 non-null  int64
35   App_Usage_Percentage                        100000 non-null  int64
36   Reminder_Emails_Clicked                     100000 non-null  int64
37   Support_Tickets_Raised                      100000 non-null  int64
38   Satisfaction_Rating                        100000 non-null  float64
39   Completed                                   100000 non-null  object
dtypes: datetime64[ns](1), float64(8), int64(15), object(16)
memory usage: 30.5+ MB

```

## Univariate Analysis

```

In [6]: Numerical_Cols = df.select_dtypes(include=['int64', 'float64']).columns
        Categorical_Cols = df.select_dtypes(include=['object', 'category', 'bool']).columns

print("Numerical columns:", Numerical_Cols.tolist())
print("Categorical columns:", Categorical_Cols.tolist())

```

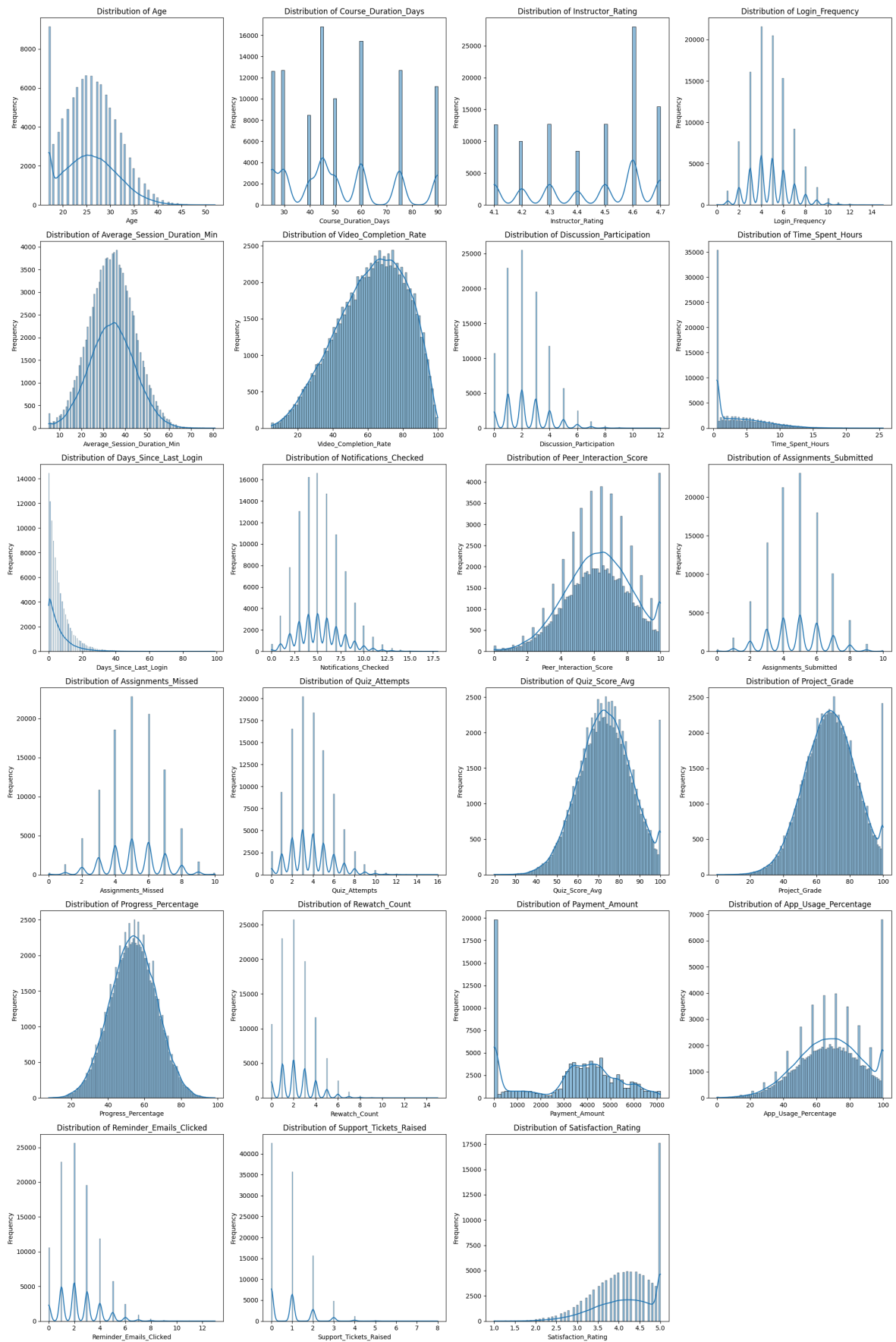
Numerical columns: ['Age', 'Course\_Duration\_Days', 'Instructor\_Rating', 'Login\_Frequency', 'Average\_Session\_Duration\_Min', 'Video\_Completion\_Rate', 'Discussion\_Participation', 'Time\_Spent\_Hours', 'Days\_Since\_Last\_Login', 'Notifications\_Checked', 'Peer\_Interaction\_Score', 'Assignments\_Submitted', 'Assignments\_Missed', 'Quiz\_Attempts', 'Quiz\_Score\_Avg', 'Project\_Grade', 'Progress\_Percentage', 'Rewatch\_Count', 'Payment\_Amount', 'App\_Usage\_Percentage', 'Reminder\_Emails\_Clicked', 'Support\_Tickets\_Raised', 'Satisfaction\_Rating']

Categorical columns: ['Student\_ID', 'Name', 'Gender', 'Education\_Level', 'Employment\_Status', 'City', 'Device\_Type', 'Internet\_Connection\_Quality', 'Course\_ID', 'Course\_Name', 'Category', 'Course\_Level', 'Payment\_Mode', 'Fee\_Paid', 'Discount\_Used', 'Completed']

```
In [8]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(20, 30))
for i, col in enumerate(Numerical_Cols):
    plt.subplot(6, 4, i + 1)
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.tight_layout()
plt.suptitle('Histograms of Numerical Features', y=1.02, fontsize=20)
plt.show()
```

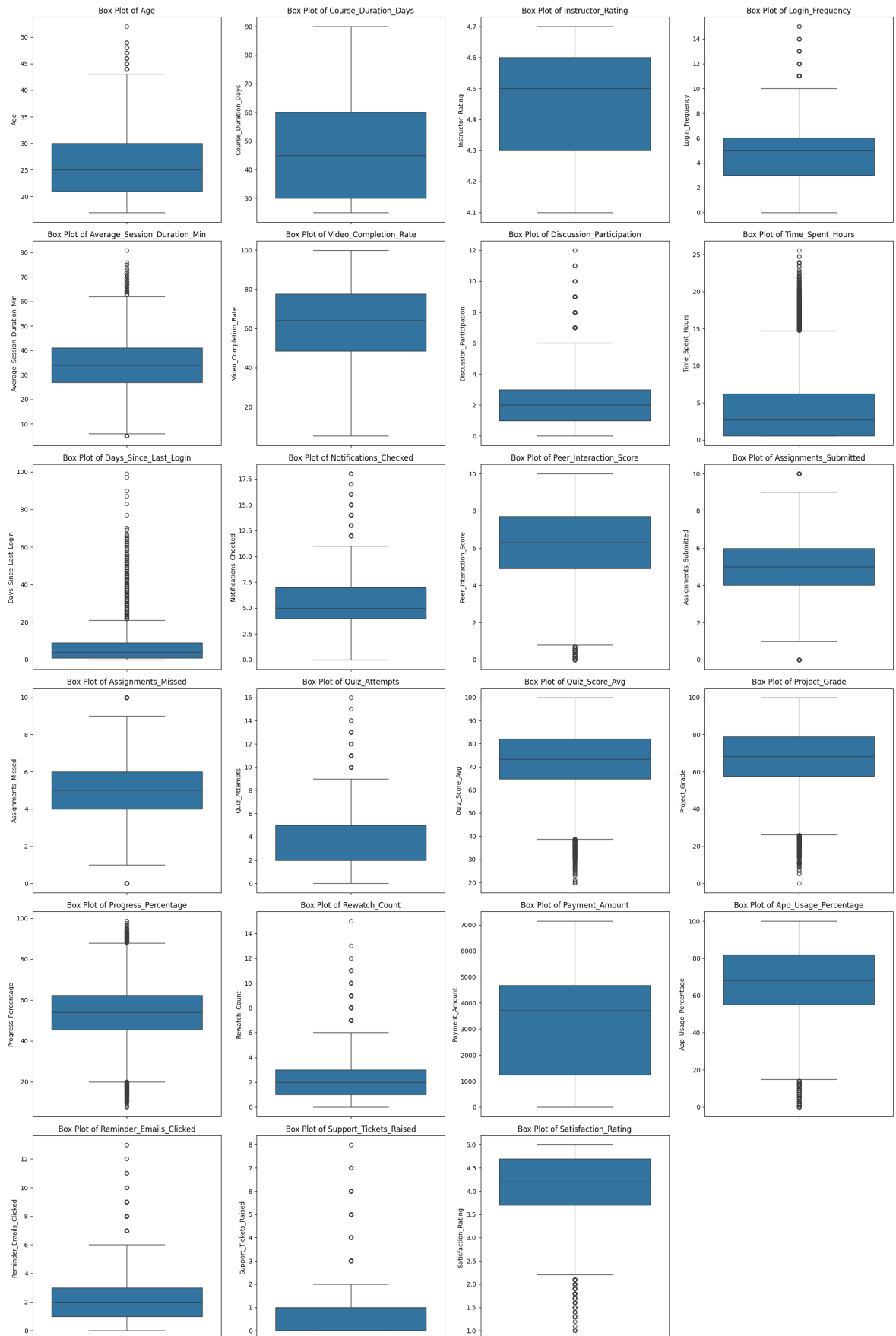
## Histograms of Numerical Features



```
In [9]: plt.figure(figsize=(20, 30))
for i, col in enumerate(Numerical_Cols):
    plt.subplot(6, 4, i + 1)
    sns.boxplot(y=df[col])
    plt.title(f'Box Plot of {col}')
```

```
plt.ylabel(col)
plt.tight_layout()
plt.suptitle('Box Plots of Numerical Features', y=1.02, fontsize=20)
plt.show()
```

Box Plots of Numerical Features

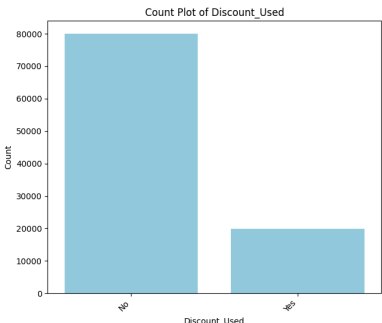
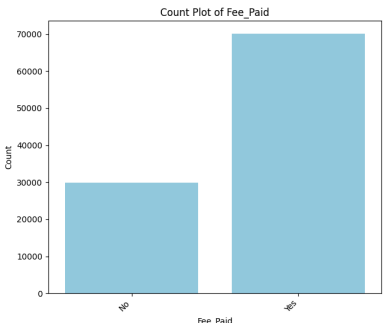
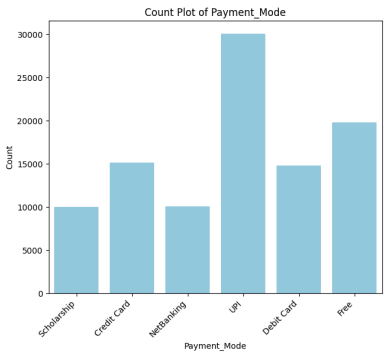
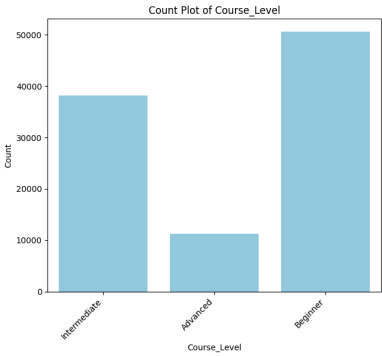
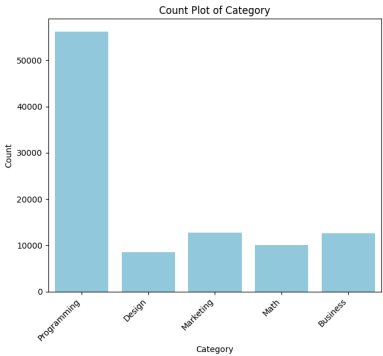
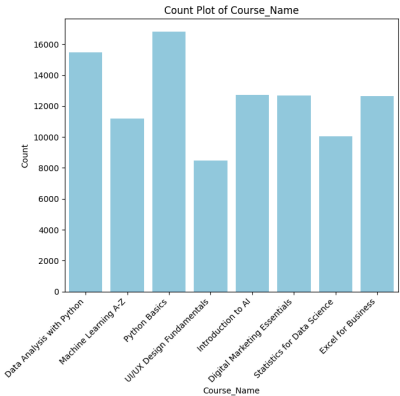
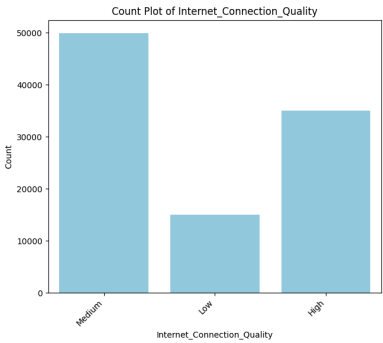
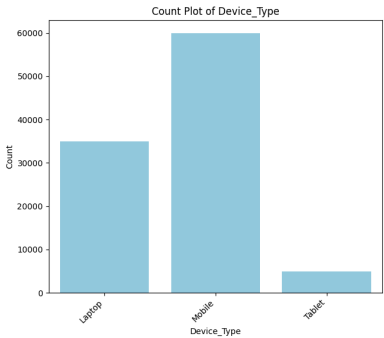
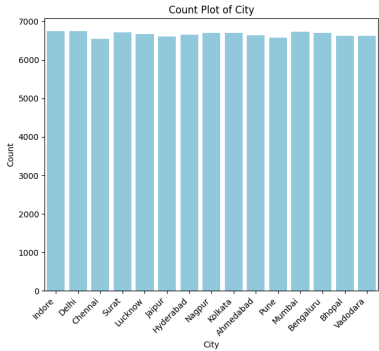
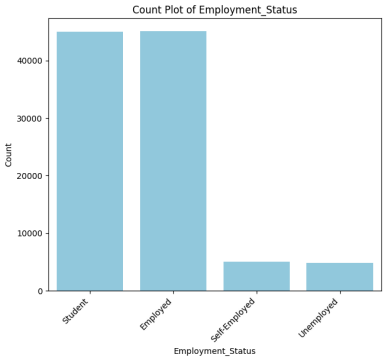
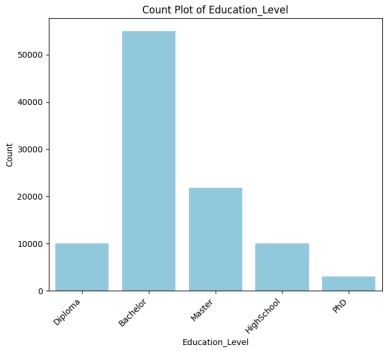
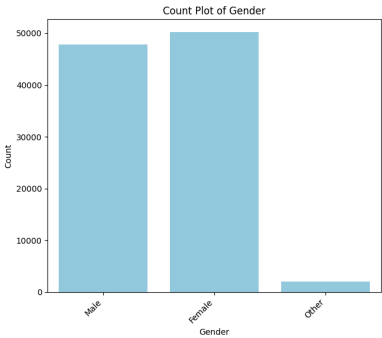


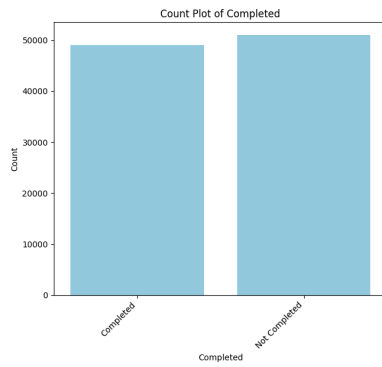
```
In [11]: plt.figure(figsize=(20, 40))
for i, col in enumerate(Categorical_Cols):
    if col not in ['Student_ID', 'Name', 'Course_ID']:
        plt.subplot(6, 3, i + 1)
        sns.countplot(data=df, x=col, color='skyblue')
        plt.title(f'Count Plot of {col}')
        plt.xlabel(col)
        plt.ylabel('Count')
        plt.xticks(rotation=45, ha='right')
        plt.tight_layout()

plt.suptitle('Count Plots of Categorical Features', y=1.02, fontsize=20)
plt.show()
```



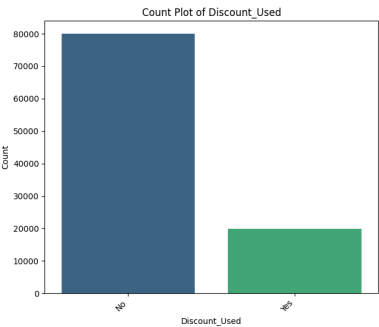
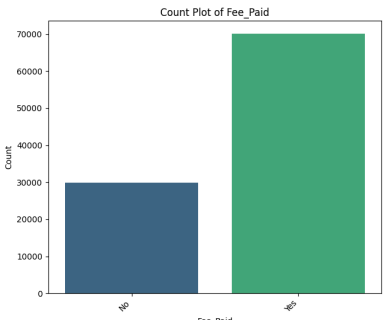
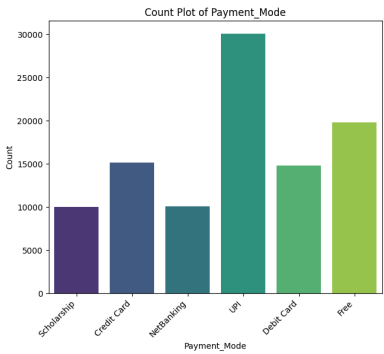
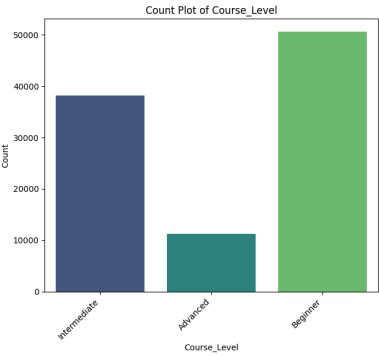
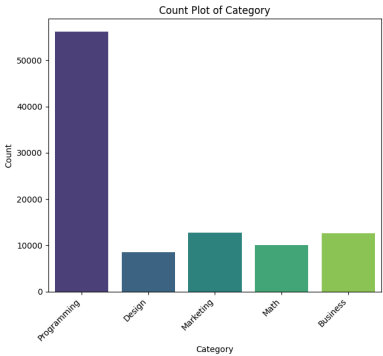
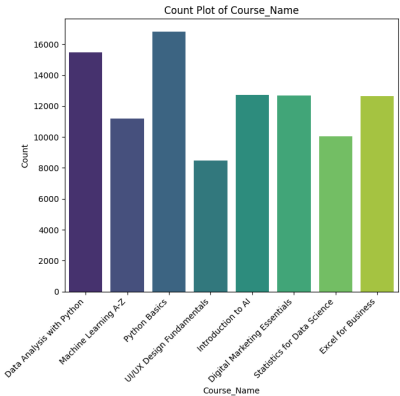
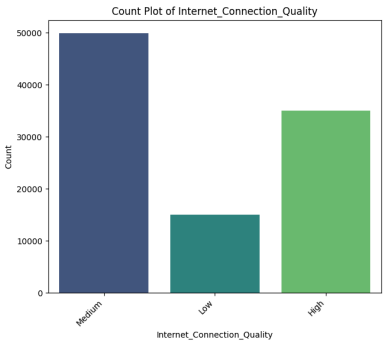
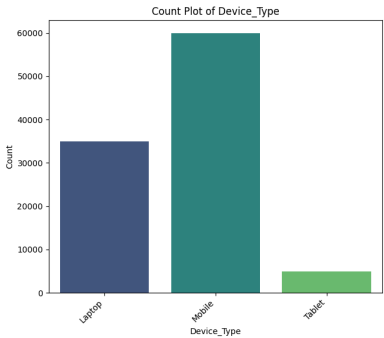
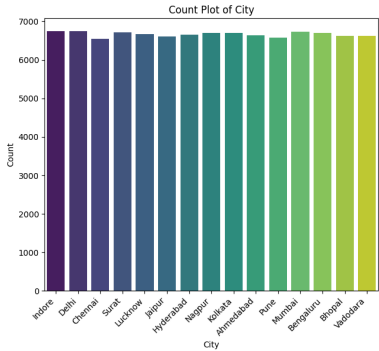
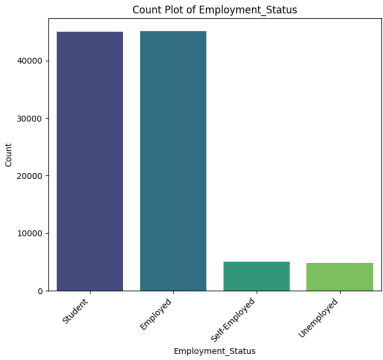
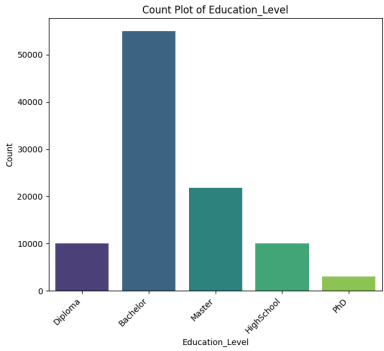
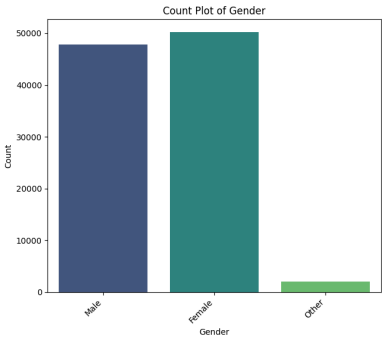
Count Plots of Categorical Features

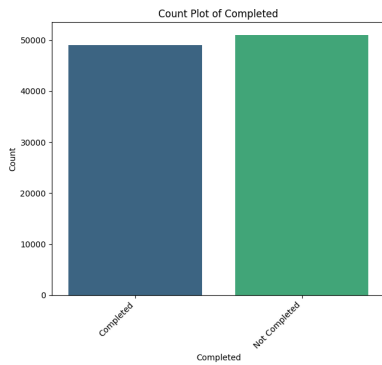




```
In [13]: plt.figure(figsize=(20, 40))
for i, col in enumerate(Categorical_Cols):
    if col not in ['Student_ID', 'Name', 'Course_ID']:
        plt.subplot(6, 3, i + 1)
        sns.countplot(data=df, x=col, hue=col, palette='viridis', legend=False)
        plt.title(f'Count Plot of {col}')
        plt.xlabel(col)
        plt.ylabel('Count')
        plt.xticks(rotation=45, ha='right')
        plt.tight_layout()
plt.suptitle('Count Plots of Categorical Features', y=1.02, fontsize=20)
plt.show()
```

Count Plots of Categorical Features





## Bivariate Analysis

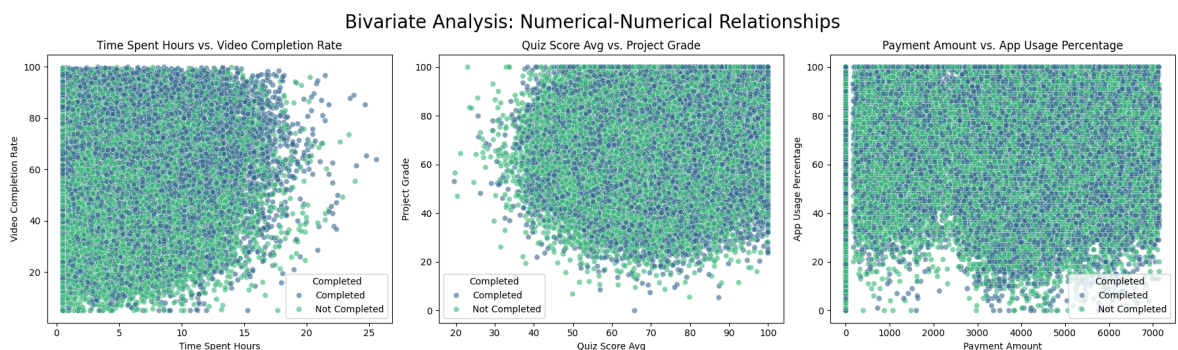
```
In [14]: plt.figure(figsize=(18, 5))

plt.subplot(1, 3, 1)
sns.scatterplot(data=df, x='Time_Spent_Hours', y='Video_Completion_Rate', hue='Completed')
plt.title('Time Spent Hours vs. Video Completion Rate')
plt.xlabel('Time Spent Hours')
plt.ylabel('Video Completion Rate')

plt.subplot(1, 3, 2)
sns.scatterplot(data=df, x='Quiz_Score_Avg', y='Project_Grade', hue='Completed',
plt.title('Quiz Score Avg vs. Project Grade')
plt.xlabel('Quiz Score Avg')
plt.ylabel('Project Grade')

plt.subplot(1, 3, 3)
sns.scatterplot(data=df, x='Payment_Amount', y='App_Usage_Percentage', hue='Completed')
plt.title('Payment Amount vs. App Usage Percentage')
plt.xlabel('Payment Amount')
plt.ylabel('App Usage Percentage')

plt.tight_layout()
plt.suptitle('Bivariate Analysis: Numerical-Numerical Relationships', y=1.05, fontweight='bold')
plt.show()
```



```
In [17]: plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)
sns.boxplot(data=df, x='Completed', y='Satisfaction_Rating', hue='Completed', palette='magma')
plt.title('Satisfaction Rating by Completion Status')
plt.xlabel('Completion Status')
```

```

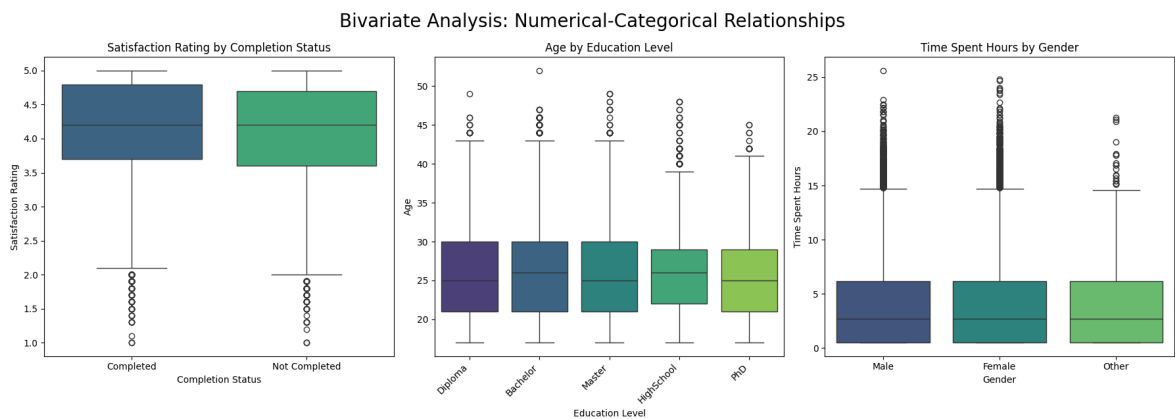
plt.ylabel('Satisfaction Rating')

plt.subplot(1, 3, 2)
sns.boxplot(data=df, x='Education_Level', y='Age', hue='Education_Level', palette=
plt.title('Age by Education Level')
plt.xlabel('Education Level')
plt.ylabel('Age')
plt.xticks(rotation=45, ha='right')

plt.subplot(1, 3, 3)
sns.boxplot(data=df, x='Gender', y='Time_Spent_Hours', hue='Gender', palette='vi
plt.title('Time Spent Hours by Gender')
plt.xlabel('Gender')
plt.ylabel('Time Spent Hours')

plt.tight_layout()
plt.suptitle('Bivariate Analysis: Numerical-Categorical Relationships', y=1.05,
plt.show()

```



```

In [18]: plt.figure(figsize=(20, 7))

plt.subplot(1, 3, 1)
crosstab_edu_comp = pd.crosstab(df['Education_Level'], df['Completed'])
crosstab_edu_comp.plot(kind='bar', stacked=True, colormap='viridis', ax=plt.gca()
plt.title('Completion Status by Education Level')
plt.xlabel('Education Level')
plt.ylabel('Count')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Completed', bbox_to_anchor=(1.05, 1), loc='upper left')

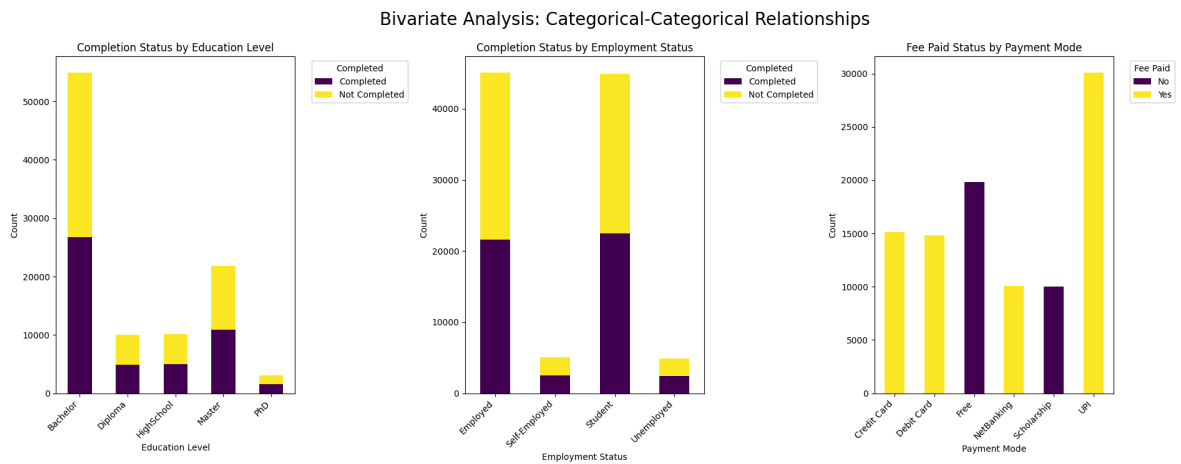
plt.subplot(1, 3, 2)
crosstab_emp_comp = pd.crosstab(df['Employment_Status'], df['Completed'])
crosstab_emp_comp.plot(kind='bar', stacked=True, colormap='viridis', ax=plt.gca()
plt.title('Completion Status by Employment Status')
plt.xlabel('Employment Status')
plt.ylabel('Count')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Completed', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.subplot(1, 3, 3)
crosstab_pay_fee = pd.crosstab(df['Payment_Mode'], df['Fee_Paid'])
crosstab_pay_fee.plot(kind='bar', stacked=True, colormap='viridis', ax=plt.gca()

```

```
plt.title('Fee Paid Status by Payment Mode')
plt.xlabel('Payment Mode')
plt.ylabel('Count')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Fee Paid', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout(rect=[0, 0, 0.95, 1])
plt.suptitle('Bivariate Analysis: Categorical-Categorical Relationships', y=1.05)
plt.show()
```



## Multivariate Analysis

```
In [20]: Correlation_Matrix = df[Numerical_Cols].corr()
print("Correlation Matrix for Numerical Features:")
print(Correlation_Matrix.head())
```

Correlation Matrix for Numerical Features:

|                              | Age       | Course_Duration_Days | \ |
|------------------------------|-----------|----------------------|---|
| Age                          | 1.000000  | -0.002204            |   |
| Course_Duration_Days         | -0.002204 | 1.000000             |   |
| Instructor_Rating            | -0.003943 | 0.638474             |   |
| Login_Frequency              | -0.002113 | 0.006298             |   |
| Average_Session_Duration_Min | -0.001146 | 0.002086             |   |

|                              | Instructor_Rating | Login_Frequency | \ |
|------------------------------|-------------------|-----------------|---|
| Age                          | -0.003943         | -0.002113       |   |
| Course_Duration_Days         | 0.638474          | 0.006298        |   |
| Instructor_Rating            | 1.000000          | -0.000815       |   |
| Login_Frequency              | -0.000815         | 1.000000        |   |
| Average_Session_Duration_Min | 0.000772          | 0.082255        |   |

|                              | Average_Session_Duration_Min | \ |
|------------------------------|------------------------------|---|
| Age                          | -0.001146                    |   |
| Course_Duration_Days         | 0.002086                     |   |
| Instructor_Rating            | 0.000772                     |   |
| Login_Frequency              | 0.082255                     |   |
| Average_Session_Duration_Min | 1.000000                     |   |

|                              | Video_Completion_Rate | Discussion_Participation | \ |
|------------------------------|-----------------------|--------------------------|---|
| Age                          | -0.003046             | 0.005319                 |   |
| Course_Duration_Days         | -0.001489             | 0.004251                 |   |
| Instructor_Rating            | -0.002161             | 0.002980                 |   |
| Login_Frequency              | 0.049121              | 0.083339                 |   |
| Average_Session_Duration_Min | 0.043143              | 0.079099                 |   |

|                              | Time_Spent_Hours | Days_Since_Last_Login | \ |
|------------------------------|------------------|-----------------------|---|
| Age                          | -0.002272        | -0.002632             |   |
| Course_Duration_Days         | 0.003659         | -0.000477             |   |
| Instructor_Rating            | 0.001767         | -0.000344             |   |
| Login_Frequency              | 0.204219         | -0.055705             |   |
| Average_Session_Duration_Min | 0.164911         | -0.039064             |   |

|                              | Notifications_Checked | ... | Quiz_Attempts | \ |
|------------------------------|-----------------------|-----|---------------|---|
| Age                          | -0.006200             | ... | -0.001572     |   |
| Course_Duration_Days         | 0.000725              | ... | -0.000185     |   |
| Instructor_Rating            | 0.001614              | ... | 0.001436      |   |
| Login_Frequency              | 0.090068              | ... | 0.088041      |   |
| Average_Session_Duration_Min | 0.082198              | ... | 0.078041      |   |

|                              | Quiz_Score_Avg | Project_Grade | \ |
|------------------------------|----------------|---------------|---|
| Age                          | 0.004824       | 0.001187      |   |
| Course_Duration_Days         | 0.005631       | 0.000451      |   |
| Instructor_Rating            | 0.005340       | 0.000315      |   |
| Login_Frequency              | 0.104183       | 0.082453      |   |
| Average_Session_Duration_Min | 0.090822       | 0.067840      |   |

|                              | Progress_Percentage | Rewatch_Count | \ |
|------------------------------|---------------------|---------------|---|
| Age                          | 0.001319            | -0.001171     |   |
| Course_Duration_Days         | 0.000582            | -0.004315     |   |
| Instructor_Rating            | -0.000188           | -0.002634     |   |
| Login_Frequency              | 0.118722            | 0.081335      |   |
| Average_Session_Duration_Min | 0.099467            | 0.072932      |   |

|                      | Payment_Amount | App_Usage_Percentage | \ |
|----------------------|----------------|----------------------|---|
| Age                  | -0.000391      | -0.003771            |   |
| Course_Duration_Days | 0.350785       | 0.002128             |   |

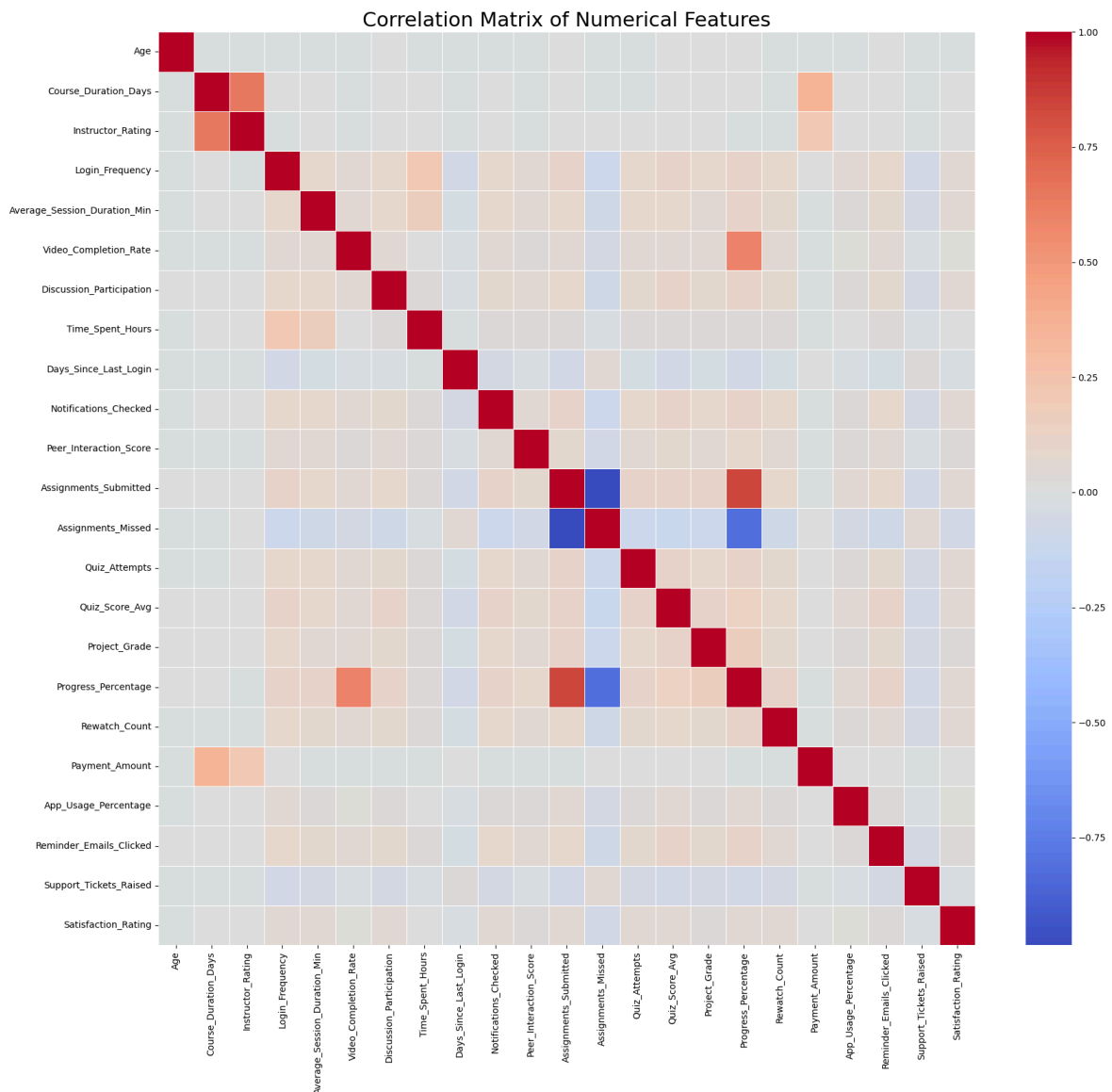
|                              |           |          |
|------------------------------|-----------|----------|
| Instructor_Rating            | 0.223238  | 0.004379 |
| Login_Frequency              | 0.002035  | 0.042886 |
| Average_Session_Duration_Min | -0.002398 | 0.036819 |

|                              | Reminder_Emails_Clicked | Support_Tickets_Raised \ |
|------------------------------|-------------------------|--------------------------|
| Age                          | 0.000846                | 0.000195                 |
| Course_Duration_Days         | 0.000356                | -0.002610                |
| Instructor_Rating            | 0.001577                | -0.003379                |
| Login_Frequency              | 0.080648                | -0.058735                |
| Average_Session_Duration_Min | 0.071708                | -0.053008                |

|                              | Satisfaction_Rating |
|------------------------------|---------------------|
| Age                          | -0.000848           |
| Course_Duration_Days         | 0.004594            |
| Instructor_Rating            | 0.002055            |
| Login_Frequency              | 0.050876            |
| Average_Session_Duration_Min | 0.041320            |

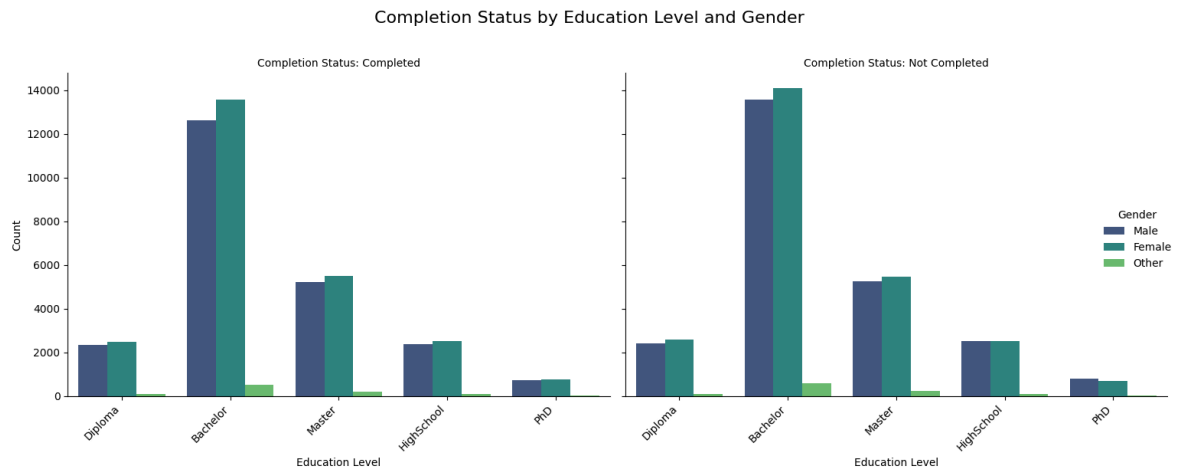
[5 rows x 23 columns]

```
In [21]: plt.figure(figsize=(20, 18))
sns.heatmap(Correlation_Matrix, annot=False, cmap='coolwarm', fmt=".2f", linewidth
plt.title('Correlation Matrix of Numerical Features', fontsize=22)
plt.show()
```

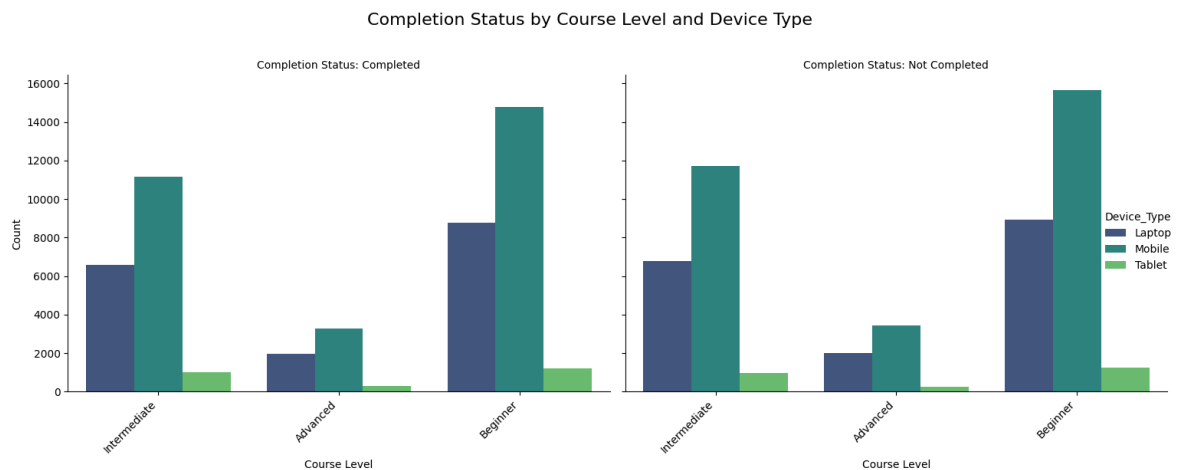




```
In [22]: g = sns.catplot(data=df, x='Education_Level', hue='Gender', col='Completed', kind='bar')
g.set_axis_labels("Education Level", "Count")
g.set_xticklabels(rotation=45, ha='right')
g.set_titles("Completion Status: {col_name}")
plt.suptitle('Completion Status by Education Level and Gender', y=1.02, fontsize=14)
plt.tight_layout()
plt.show()
```



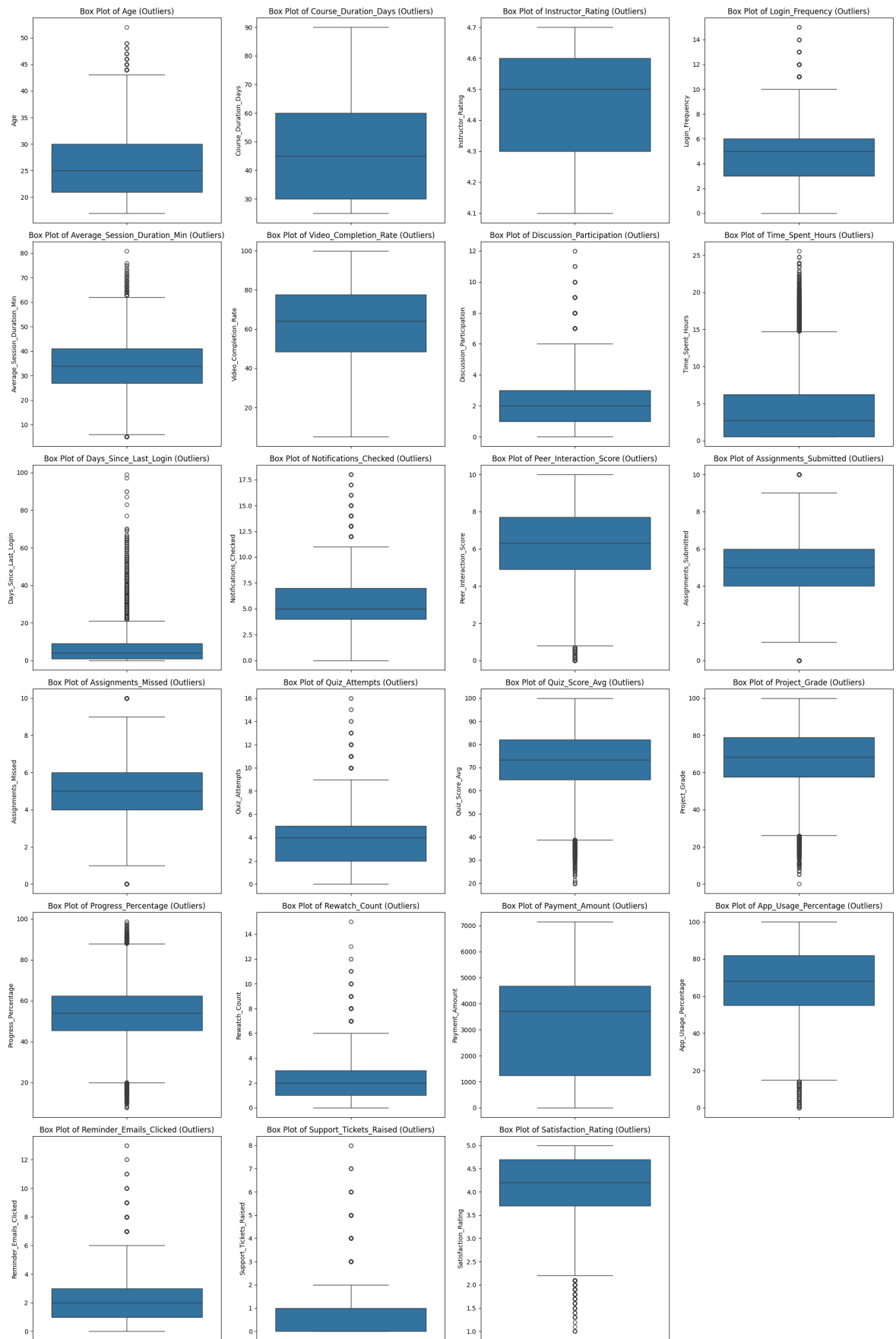
```
In [23]: g = sns.catplot(data=df, x='Course_Level', hue='Device_Type', col='Completed', kind='bar')
g.set_axis_labels("Course Level", "Count")
g.set_xticklabels(rotation=45, ha='right')
g.set_titles("Completion Status: {col_name}")
plt.suptitle('Completion Status by Course Level and Device Type', y=1.02, fontsize=14)
plt.tight_layout()
plt.show()
```



## Outlier Visualization

```
In [25]: plt.figure(figsize=(20, 30))
for i, col in enumerate(Numerical_Cols):
    plt.subplot(6, 4, i + 1)
    sns.boxplot(y=df[col])
    plt.title(f'Box Plot of {col} (Outliers)')
    plt.ylabel(col)
    plt.tight_layout()
plt.suptitle('Outlier Visualization: Box Plots of Numerical Features', y=1.02, fontsize=14)
plt.show()
```

## Outlier Visualization: Box Plots of Numerical Features



## Outlier Analysis

In [27]: `Key_Numerical_Features = ['Age', 'Time_Spent_Hours', 'Satisfaction_Rating', 'Pay`

```
print("\nOutlier Quantification using IQR Method:")
for col in Key_Numerical_Features:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    Outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    Num_Outliers = len(Outliers)
    Percentage_Outliers = (Num_Outliers / len(df)) * 100

    print(f"\nFeature: {col}")
    print(f"   Q1: {Q1:.2f}")
    print(f"   Q3: {Q3:.2f}")
    print(f"   IQR: {IQR:.2f}")
    print(f"   Lower Bound: {lower_bound:.2f}")
    print(f"   Upper Bound: {upper_bound:.2f}")
    print(f"   Number of Outliers: {Num_Outliers}")
    print(f"   Percentage of Outliers: {Percentage_Outliers:.2f}%")
```

## Outlier Quantification using IQR Method:

### Feature: Age

Q1: 21.00  
Q3: 30.00  
IQR: 9.00  
Lower Bound: 7.50  
Upper Bound: 43.50  
Number of Outliers: 147  
Percentage of Outliers: 0.15%

### Feature: Time\_Spent\_Hours

Q1: 0.50  
Q3: 6.20  
IQR: 5.70  
Lower Bound: -8.05  
Upper Bound: 14.75  
Number of Outliers: 1102  
Percentage of Outliers: 1.10%

### Feature: Satisfaction\_Rating

Q1: 3.70  
Q3: 4.70  
IQR: 1.00  
Lower Bound: 2.20  
Upper Bound: 6.20  
Number of Outliers: 582  
Percentage of Outliers: 0.58%

### Feature: Payment\_Amount

Q1: 1242.00  
Q3: 4685.00  
IQR: 3443.00  
Lower Bound: -3922.50  
Upper Bound: 9849.50  
Number of Outliers: 0  
Percentage of Outliers: 0.00%

### Feature: Support\_Tickets\_Raised

Q1: 0.00  
Q3: 1.00  
IQR: 1.00  
Lower Bound: -1.50  
Upper Bound: 2.50  
Number of Outliers: 6193  
Percentage of Outliers: 6.19%

### Feature: Login\_Frequency

Q1: 3.00  
Q3: 6.00  
IQR: 3.00  
Lower Bound: -1.50  
Upper Bound: 10.50  
Number of Outliers: 374  
Percentage of Outliers: 0.37%

### Feature: Video\_Completion\_Rate

Q1: 48.50  
Q3: 77.50  
IQR: 29.00

Lower Bound: 5.00  
Upper Bound: 121.00  
Number of Outliers: 0  
Percentage of Outliers: 0.00%

Feature: Quiz\_Score\_Avg

Q1: 64.70  
Q3: 82.00  
IQR: 17.30  
Lower Bound: 38.75  
Upper Bound: 107.95  
Number of Outliers: 311  
Percentage of Outliers: 0.31%

Feature: Project\_Grade

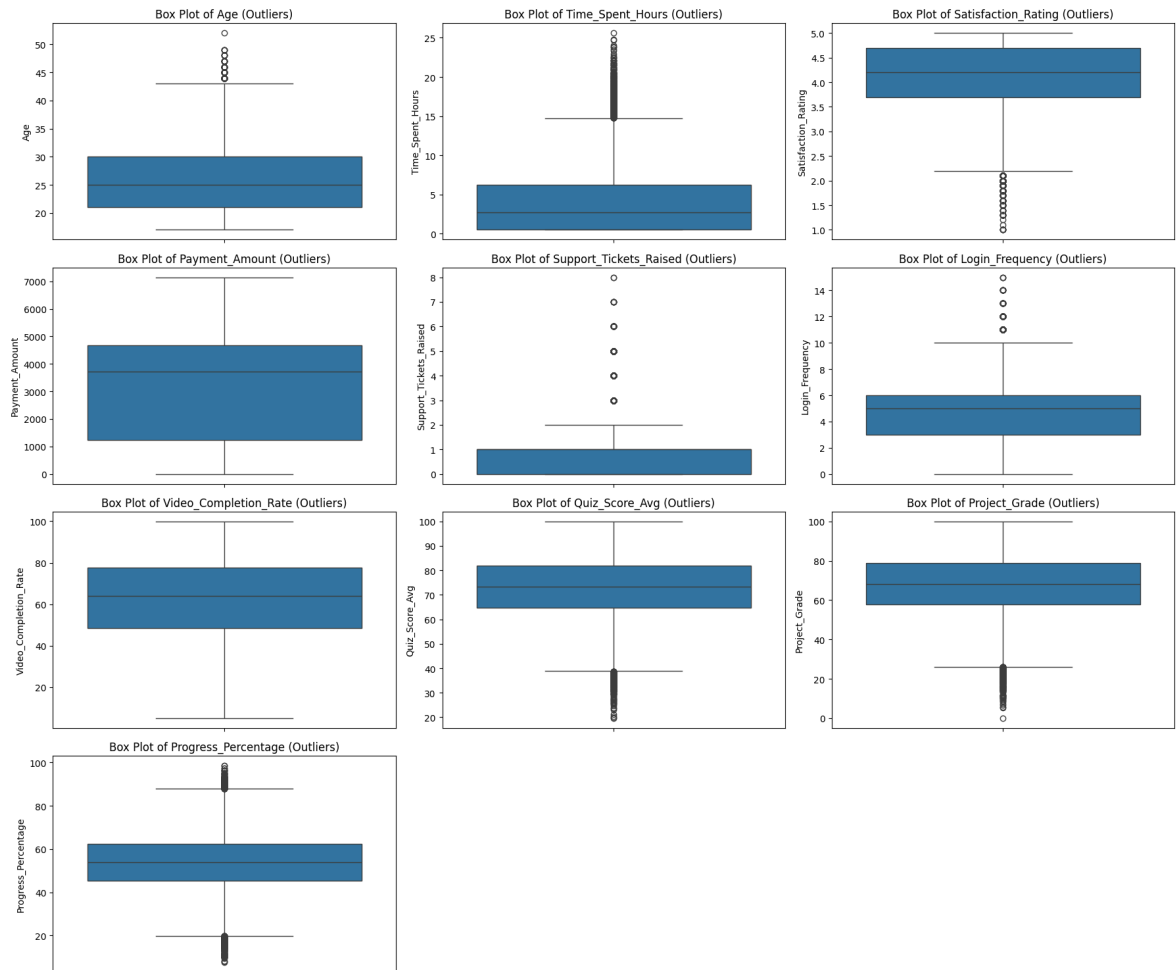
Q1: 57.70  
Q3: 78.80  
IQR: 21.10  
Lower Bound: 26.05  
Upper Bound: 110.45  
Number of Outliers: 356  
Percentage of Outliers: 0.36%

Feature: Progress\_Percentage

Q1: 45.40  
Q3: 62.40  
IQR: 17.00  
Lower Bound: 19.90  
Upper Bound: 87.90  
Number of Outliers: 466  
Percentage of Outliers: 0.47%

```
In [29]: plt.figure(figsize=(18, 15))
for i, col in enumerate(Key_Numerical_Features):
    plt.subplot(4, 3, i + 1)
    sns.boxplot(y=df[col])
    plt.title(f'Box Plot of {col} (Outliers)')
    plt.ylabel(col)
    plt.tight_layout()
plt.suptitle('Outlier Visualization for Key Numerical Features', y=1.02, fontsize=14)
plt.show()
```

## Outlier Visualization for Key Numerical Features



## Feature Engineering for Completion Prediction

```
In [30]: df['Engagement_Score'] = df['Login_Frequency'] * df['Average_Session_Duration_Mi
df['Learning_Effectiveness'] = df['Video_Completion_Rate'] * df['Quiz_Score_Avg']

df['Assignment_Success_Rate'] = df['Assignments_Submitted'] / (df['Assignments_S
df['Assignment_Success_Rate'] = df['Assignment_Success_Rate'].fillna(0)

print("New features 'Engagement_Score', 'Learning_Effectiveness', and 'Assignmen
df.head()
```

New features 'Engagement\_Score', 'Learning\_Effectiveness', and 'Assignment\_Success\_Rate' created.

Out[30]:

|   | Student_ID | Name            | Gender | Age | Education_Level | Employment_Status | City    |
|---|------------|-----------------|--------|-----|-----------------|-------------------|---------|
| 0 | STU100000  | Vihaan Patel    | Male   | 19  | Diploma         | Student           | Indore  |
| 1 | STU100001  | Arjun Nair      | Female | 17  | Bachelor        | Student           | Delhi   |
| 2 | STU100002  | Aditya Bhardwaj | Female | 34  | Master          | Student           | Chennai |
| 3 | STU100003  | Krishna Singh   | Female | 29  | Diploma         | Employed          | Surat   |
| 4 | STU100004  | Krishna Nair    | Female | 19  | Master          | Self-Employed     | Lucknow |

5 rows × 43 columns

## Machine Learning

```
In [31]: y = df['Completed']
X = df.drop('Completed', axis=1)

Categorical_Cols_To_Encode = [col for col in X.select_dtypes(include=['object'],
Numerical_Cols_To_Scale = X.select_dtypes(include=['int64', 'float64']).columns

print("Target variable 'y' created.")
print("Features 'X' created (excluding 'Completed').")
print("Categorical columns identified for encoding:", Categorical_Cols_To_Encode)
print("Numerical columns identified for scaling:", Numerical_Cols_To_Scale.tolist())
```

Target variable 'y' created.  
Features 'X' created (excluding 'Completed').  
Categorical columns identified for encoding: ['Gender', 'Education\_Level', 'Employment\_Status', 'City', 'Device\_Type', 'Internet\_Connection\_Quality', 'Course\_Name', 'Category', 'Course\_Level', 'Payment\_Mode', 'Fee\_Paid', 'Discount\_Used']  
Numerical columns identified for scaling: ['Age', 'Course\_Duration\_Days', 'Instructor\_Rating', 'Login\_Frequency', 'Average\_Session\_Duration\_Min', 'Video\_Completion\_Rate', 'Discussion\_Participation', 'Time\_Spent\_Hours', 'Days\_Since\_Last\_Login', 'Notifications\_Checked', 'Peer\_Interaction\_Score', 'Assignments\_Submitted', 'Assignments\_Missed', 'Quiz\_Attempts', 'Quiz\_Score\_Avg', 'Project\_Grade', 'Progress\_Percentage', 'Rewatch\_Count', 'Payment\_Amount', 'App\_Usage\_Percentage', 'Reminder\_Emails\_Clicked', 'Support\_Tickets\_Raised', 'Satisfaction\_Rating', 'Engagement\_Score', 'Learning\_Effectiveness', 'Assignment\_Success\_Rate']

```
In [33]: from sklearn.preprocessing import StandardScaler

X_encoded = pd.get_dummies(X[Categorical_Cols_To_Encode], drop_first=True)
print("Categorical columns one-hot encoded.")

Scaler = StandardScaler()
X_scaled = Scaler.fit_transform(X[Numerical_Cols_To_Scale])
X_scaled = pd.DataFrame(X_scaled, columns=Numerical_Cols_To_Scale, index=X.index)
print("Numerical columns scaled.")
```

```
X_processed = pd.concat([X_scaled, X_encoded], axis=1)
print("Scaled numerical features and one-hot encoded categorical features combin
print("Shape of X_processed:", X_processed.shape)
X_processed.head()
```

Categorical columns one-hot encoded.

Numerical columns scaled.

Scaled numerical features and one-hot encoded categorical features combined.

Shape of X\_processed: (100000, 73)

Out[33]:

|   | Age       | Course_Duration_Days | Instructor_Rating | Login_Frequency | Average_Session |
|---|-----------|----------------------|-------------------|-----------------|-----------------|
| 0 | -1.194884 | 0.402599             | 1.261028          | -0.965968       |                 |
| 1 | -1.551056 | 1.878635             | 0.767517          | -0.424925       |                 |
| 2 | 1.476406  | -0.335419            | 0.767517          | 0.116119        |                 |
| 3 | 0.585976  | -0.581426            | -0.219504         | -1.507012       |                 |
| 4 | -1.194884 | 1.878635             | 0.767517          | -1.507012       |                 |

5 rows × 73 columns



In [34]: `from sklearn.model_selection import train_test_split`

```
y_encoded = y.map({'Not Completed': 0, 'Completed': 1})
```

```
X_train, X_test, y_train, y_test = train_test_split(X_processed, y_encoded, test
```

```
print("Data split into training and testing sets.")
```

```
print(f"X_train shape: {X_train.shape}")
```

```
print(f"X_test shape: {X_test.shape}")
```

```
print(f"y_train shape: {y_train.shape}")
```

```
print(f"y_test shape: {y_test.shape}")
```

Data split into training and testing sets.

X\_train shape: (80000, 73)

X\_test shape: (20000, 73)

y\_train shape: (80000,)

y\_test shape: (20000,)

## Train and Evaluate Classification Models

```
In [35]: from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, f1_score

print("Required libraries imported successfully.")
```

Required libraries imported successfully.

```
In [36]: def train_evaluate_model(model, X_train, y_train, X_test, y_test, model_name):
print(f"\n--- Evaluating {model_name} ---")
```



```

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(classification_report(y_test, y_pred))
    f1 = f1_score(y_test, y_pred, pos_label=1)
    return f1

print("Train_Evaluate_Model Function Defined.")

```

Train\_Evaluate\_Model Function Defined.

```

In [37]: Logistic_Regression_Model = LogisticRegression(random_state=42, solver='liblinear')
f1_lr = train_evaluate_model(Logistic_Regression_Model, X_train, y_train, X_test, y_test)

print(f"Logistic Regression F1-score: {f1_lr:.4f}")

```

```

--- Evaluating Logistic Regression ---
              precision    recall  f1-score   support

      0       0.61      0.62      0.62     10194
      1       0.60      0.59      0.59      9806

 accuracy          0.60
macro avg          0.60      0.60      0.60     20000
weighted avg       0.60      0.60      0.60     20000

```

Logistic Regression F1-score: 0.5921

```

In [38]: Decision_Tree_Model = DecisionTreeClassifier(random_state=42)
f1_dt = train_evaluate_model(Decision_Tree_Model, X_train, y_train, X_test, y_test)

print(f"Decision Tree Classifier F1-score: {f1_dt:.4f}")

```

```

--- Evaluating Decision Tree Classifier ---
              precision    recall  f1-score   support

      0       0.54      0.53      0.54     10194
      1       0.52      0.52      0.52      9806

 accuracy          0.53
macro avg          0.53      0.53      0.53     20000
weighted avg       0.53      0.53      0.53     20000

```

Decision Tree Classifier F1-score: 0.5198

```

In [39]: Random_Forest_Model = RandomForestClassifier(random_state=42, n_estimators=100)
f1_rf = train_evaluate_model(Random_Forest_Model, X_train, y_train, X_test, y_test)

print(f"Random Forest Classifier F1-score: {f1_rf:.4f}")

```

```

--- Evaluating Random Forest Classifier ---
              precision    recall  f1-score   support

      0       0.60      0.63      0.61     10194
      1       0.59      0.57      0.58      9806

 accuracy          0.60
macro avg          0.60      0.60      0.60     20000
weighted avg       0.60      0.60      0.60     20000

```

Random Forest Classifier F1-score: 0.5811

```
In [40]: print("\n--- Model Performance Comparison (F1-score for positive class) ---")
print(f"Logistic Regression F1-score: {f1_lr:.4f}")
print(f"Decision Tree Classifier F1-score: {f1_dt:.4f}")
print(f"Random Forest Classifier F1-score: {f1_rf:.4f}")

best_f1 = max(f1_lr, f1_dt, f1_rf)

if best_f1 == f1_lr:
    best_model_name = "Logistic Regression"
elif best_f1 == f1_dt:
    best_model_name = "Decision Tree Classifier"
else:
    best_model_name = "Random Forest Classifier"

print(f"\nThe best performing model based on F1-score is: {best_model_name} with

--- Model Performance Comparison (F1-score for positive class) ---
Logistic Regression F1-score: 0.5921
Decision Tree Classifier F1-score: 0.5198
Random Forest Classifier F1-score: 0.5811
```

The best performing model based on F1-score is: Logistic Regression with an F1-score of 0.5921

## Explanation of Model Performance

From the evaluation, the **Logistic Regression** model emerged as the best-performing model with an F1-score of 0.5921 for the positive class ('Completed'). This indicates that, for this specific dataset and the features engineered, a simpler linear model was more effective at balancing precision and recall for predicting course completion compared to the other models tested.

### Why Logistic Regression Performed Best (in this instance):

1. **Simplicity and Interpretability:** Logistic Regression is a linear model that estimates the probability of a binary outcome. Its simplicity can be an advantage when the relationship between features and the target variable is largely linear or when the noise in the data is significant enough to hinder more complex models.
2. **Robustness to Outliers (with scaling):** While Logistic Regression can be sensitive to outliers, the numerical features were scaled using `StandardScaler` during preprocessing. This likely helped in mitigating the negative impact of outliers, allowing the model to find a more stable decision boundary.
3. **Well-Separated Classes (to some extent):** The F1-score indicates that the classes ('Completed' vs. 'Not Completed') are reasonably separable by a linear boundary based on the processed features. The engineered features such as 'Engagement\_Score' and 'Learning\_Effectiveness' might have captured linear trends that Logistic Regression can effectively leverage.

### Addressing the Random Forest Classifier:

The **Random Forest Classifier** achieved an F1-score of 0.5811, which is slightly lower than that of Logistic Regression. While Random Forests are powerful ensemble methods known for their ability to handle complex, non-linear relationships and reduce overfitting, there could be several reasons why it didn't outperform Logistic Regression in this particular scenario:

1. **Data Characteristics:** The relationships within this dataset might be predominantly linear or piecewise linear, which Logistic Regression can model efficiently. Random Forests, despite their capability for complexity, might not find significantly better non-linear patterns if they don't exist prominently.
2. **Hyperparameter Tuning:** The Random Forest was trained with `n_estimators=100` and `random_state=42`. While `n_estimators=100` is a reasonable starting point, the model's performance could potentially be improved with more extensive hyperparameter tuning (e.g., `max_depth`, `min_samples_split`, `min_samples_leaf`) using techniques like GridSearchCV or RandomizedSearchCV.
3. **Feature Importance:** It's possible that while Random Forest implicitly performs feature selection, the specific combination of features and their interactions, as presented, didn't provide a substantial non-linear advantage over the linear decision boundary found by Logistic Regression.
4. **Overfitting (less likely here):** Random Forests are generally robust to overfitting, but without cross-validation or more aggressive tuning, there's always a slight chance. However, given its performance is close to Logistic Regression, it's more likely a matter of the data's underlying structure favoring a simpler model for the chosen metrics.

In conclusion, while Random Forest is a strong contender, for this specific dataset and current feature set, Logistic Regression proved to be marginally more effective in predicting course completion based on the F1-score.

## Summary

- The dataset, comprising 100,000 entries and 40 columns, was loaded successfully with no missing values or duplicate rows identified. The `Enrollment_Date` column was correctly converted to a datetime format.
- Univariate analysis provided individual distributions for all numerical features (histograms and box plots) and categorical features (count plots), establishing a foundational understanding of each variable.
- Bivariate analysis explored relationships:
  - Scatter plots revealed potential correlations between numerical pairs such as `Time_Spent_Hours` and `Video_Completion_Rate`, `Quiz_Score_Avg` and `Project_Grade`, and `Payment_Amount` and `App_Usage_Percentage`, often visualized in relation to course completion status.
  - Box plots highlighted differences in numerical variables across categorical groups, for example, `Satisfaction_Rating` by completion status, `Age` by

Education\_Level , and Time\_Spent\_Hours by Gender .

- Stacked bar plots examined relationships between categorical variables, such as the distribution of Completed status across different Education\_Level and Employment\_Status categories, and Fee\_Paid status by Payment\_Mode .
- Multivariate analysis identified complex interactions:
  - A correlation heatmap of numerical features showed linear relationships, notably a moderate positive correlation of 0.638 between Course\_Duration\_Days and Instructor\_Rating .
  - Grouped bar charts investigated the combined impact of Education\_Level and Gender on Completed status, and Course\_Level and Device\_Type on Completed status, offering insights into potential demographic and usage patterns related to completion.
- Outlier visualization through box plots for all numerical features provided a clear view of data spread and identified potential outliers, which may warrant further investigation.
- **Outlier Quantification:** Using the Interquartile Range (IQR) method, outliers were quantified across ten key numerical features. Support\_Tickets\_Raised exhibited the highest percentage of outliers at 6.19%, followed by Time\_Spent\_Hours (1.10%) and Satisfaction\_Rating (0.58%). Features like Payment\_Amount and Video\_Completion\_Rate showed no outliers.
- **Impact and Handling of Outliers:** Outliers can lead to skewed statistics, model bias, reduced model accuracy, and misinterpretation. Strategies for handling them include removal (for sparse, erroneous outliers), transformation (for skewed distributions), winsorization (for moderate outlier percentages, e.g., for Support\_Tickets\_Raised ), imputation, and the use of robust models.
- **Feature Engineering:** Three new features were created to enhance predictive power:
  - Engagement\_Score : calculated as Login\_Frequency multiplied by Average\_Session\_Duration\_Min .
  - Learning\_Effectiveness : derived from Video\_Completion\_Rate multiplied by Quiz\_Score\_Avg .
  - Assignment\_Success\_Rate : computed as Assignments\_Submitted divided by the sum of Assignments\_Submitted and Assignments\_Missed , with division-by-zero cases handled by setting the rate to 0.
- **Data Preprocessing for Machine Learning:**
  - Categorical features such as 'Gender', 'Education\_Level', and 'Course\_Name' were one-hot encoded.
  - Numerical features, including 'Age', 'Course\_Duration\_Days', and 'Instructor\_Rating', were scaled using StandardScaler .
  - The processed dataset ( X\_processed ) resulted in 73 features.
  - The data was split into training and testing sets (80/20 ratio) using train\_test\_split with stratification to maintain class distribution, resulting in X\_train (80,000 samples) and X\_test (20,000 samples).
- **Model Performance Comparison:** Three classification models were trained and evaluated using the F1-score for predicting course completion:
  - **Logistic Regression:** Achieved an F1-score of 0.5921.

- **Random Forest Classifier:** Achieved an F1-score of 0.5811.
- **Decision Tree Classifier:** Achieved an F1-score of 0.5198.
- **Best Model Selection:** The **Logistic Regression** model emerged as the best-performing model with the highest F1-score of 0.5921. Its effectiveness is attributed to its simplicity, interpretability, and robustness in this scenario, likely benefiting from the pre-scaling of numerical features. Despite its power, the Random Forest Classifier did not outperform Logistic Regression, potentially due to the nature of the data favoring linear relationships or the lack of extensive hyperparameter tuning for the Random Forest.

**END**

In [ ]: